LM Software

Configuration and Cloning as Application Development

Registry: [HOWL-VDR-24-2026]

Series Path: [HOWL-VDR-1-2026] → [HOWL-VDR-2-2026] → [HOWL-MATH-3-2026] → [HOWL-MATH-4-2026] → … → [HOWL-VDR-14-2026] → … → [HOWL-VDR-21-2026] → [HOWL-VDR-22-2026] → [HOWL-VDR-23-2026] → [HOWL-VDR-24-2026]

DOI: 10.5281/zenodo.zzz

Date: May 2026

Domain: Computer Architecture / Adaptive Precision Arithmetic

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Opus 4.6.

Abstract

This paper defines LM Software — a new category of software where applications are developed by configuring language model sessions with structured state, encoding logic as Prolog rules over exact arithmetic primitives, validating behavior interactively, snapshotting working state as deployable artifacts, and cloning those artifacts on demand as running instances. The language model is the runtime. A hierarchical knowledge base tree is the address space. Prolog is the programming language. Snapshots are the binaries. Usage improves the application without rebuilding it.

LM Software is distinct from conventional software (compiled code, static behavior, developer-built), from AI tooling (LLMs wrapped in conventional orchestration code), and from conventional LLM applications (stateless models behind prompt templates). It is software developed by users through conversation, deployed as frozen state snapshots, and improved by usage through rule accumulation — where every session deposits reusable logic that makes future sessions cheaper and more capable.

This paper introduces all necessary concepts from the VDR-LLM-Prolog architecture [VDR-14] so that no prior reading is required, then develops the theory and practice of LM Software through two complete worked examples.

---

1. Why a New Category

1.1 The Conventional LLM Problem

A conventional large language model stores everything it knows as statistical associations between tokens and weights in a single undifferentiated parameter matrix [VDR-14, §1]. Every fact, every rule, every policy, every piece of domain knowledge is compressed into the same shared weights. Nothing is addressable. Nothing is independently queryable. Nothing can be updated without risking interference with everything else.

When a conventional LLM is deployed as a chatbot, its knowledge of your product, your policies, your phone numbers, and your business rules exists as statistical tendencies distributed across billions of parameters. It generates answers by predicting what tokens are likely to follow the conversation so far. Your phone number isn't retrieved from a record — it's reconstructed from token co-occurrence patterns. Your return policy isn't evaluated against rules — it's generated as plausible-sounding prose. The model might get these right most of the time. It will sometimes get them wrong, and there is no structural mechanism to prevent this.

The industry response has been to wrap LLMs in conventional software. Retrieval-augmented generation (RAG) retrieves documents and pastes them into the prompt. Function calling lets the model invoke external tools. Frameworks like LangChain orchestrate multi-step workflows in Python code. These approaches treat the LLM as a component inside conventional software — the orchestration, state management, and reliability guarantees come from the code a developer wrote around the model.

This works, but it requires conventional software development. Someone must write the retrieval pipeline, the function definitions, the orchestration logic, the error handling, the state management, the deployment infrastructure. The LLM generates text. Everything else is code.

1.2 What LM Software Is

LM Software eliminates the wrapping layer. The application is the configured LLM session itself — its state, its rules, its data, its constraints. There is no separate orchestration code because the orchestration is Prolog rules stored inside the system. There is no external database because the data lives in a hierarchical knowledge base tree that the LLM operates within. There is no deployment pipeline because deployment is cloning a snapshot.

Three properties define the category:

Developed through conversation, not code. The user interacts with an LLM session, shapes its state, tests it against inputs, writes rules when patterns emerge, iterates until the session handles its task correctly. The conversation is the development environment.

Deployed as snapshots, not containers. When the session works, the user freezes its state. That frozen state is the application. Every deployment is a clone of that snapshot — identical starting state, independent execution, governed by the same structural rules.

Improved by usage, not releases. As the application handles tasks, it encodes new patterns as Prolog rules. Each rule makes future tasks cheaper. The application gets better at its job by doing its job, within security constraints that ensure self-improvement cannot compromise the system.

1.3 What LM Software Is Not

LM Software does not replace all conventional software. Compute-intensive applications, latency-critical systems, hardware-interfacing code, and real-time control systems remain conventional software. LM Software targets judgment-heavy, data-rich, policy-governed, multi-step workflows where the value is in correctly applying rules to varied inputs: customer support, compliance review, incident investigation, document processing, triage, analysis. Where conventional software requires a developer to anticipate every case in code, LM Software handles novel cases through LLM judgment and encodes the resolution as a rule for next time.

---

2. The Structural Foundation

LM Software depends on four architectural components that distinguish it from conventional LLM applications. Each is summarized here; full specifications are in [VDR-14].

2.1 Exact Arithmetic: The VDR Triple

Every number in the system is three integers: Value, Denominator, Remainder [VDR-1]. V and D are plain integers forming an exact rational number V/D. R is the remainder — not rounding error, but exact structural information about what the denominator frame could not absorb. When R is zero, the value is an exact rational. When R is nonzero, the value carries exact structure beyond the rational frame.

The system fixes the denominator at 2^335 (called Q335), derived from continued fraction convergents of Euler's number [VDR-4/MATH-4]. This gives 100 decimal digits of precision — 10^66 times below the Planck length — for all arithmetic operations. Addition of two values is one integer addition. Multiplication is one integer multiply followed by a bit extraction that separates the quotient from the remainder. The denominator never grows. Overflow goes to remainder tree depth, not denominator magnitude.

This matters for LM Software because every value the system stores, computes, or compares is exact. A confidence score is an exact fraction 85/100, not a float approximation. A threshold comparison is exact integer arithmetic, not floating-point "close enough." Policy rules evaluate to exactly yes or exactly no.

2.2 Knowledge Bases: The Address Space

A knowledge base (KB) is a structured container with 26 typed fields organized in five groups [VDR-5, VDR-8, VDR-14]:

Identity: name, dotted path (root.org.acme.support), integer ID assigned sequentially and never reused.

Persistent state: facts (predicate with typed arguments and provenance), rules (Prolog head-body implications), constraints (structured invariants with scope and violation policy), connections (typed relationships to other KBs), grammars (bidirectional templates for generating and parsing structured data), IOSE declarations (interface contracts).

Live state: working data (scoped variable bindings), LRU caches, counters, locks, queues, stacks, ring buffers, and bitsets. All bounded with declared maximum capacity. Live state is cleared by reset and captured by snapshot. Persistent state survives everything.

Structural: parent ID, children IDs, mounts (cross-branch references).

Metadata: visibility (public, internal, owner-only), frozen flag, owner, timestamps.

KBs form a tree. Every KB has at most one parent and any number of children. The root KB has no parent. The entire system — data, rules, user accounts, application state, deployment configuration — lives in one tree.

Humans address KBs by dotted paths: root.products.myapp.support. The runtime addresses them by integer IDs. A hash map connects paths to IDs, resolved once per turn and cached. All subsequent operations use the integer. Access to any data anywhere in the system is two integers: KB ID plus slot ID. Constant time.

2.3 Prolog: The Logic Layer

The Prolog engine is typed structs, not a language runtime [VDR-5, VDR-14]. Facts are predicates with ordered typed arguments and provenance — source KB, turn number, derivation record. Rules are head-body implications: the head pattern matches a query, the body goals are evaluated recursively. Depth-first search with backtracking, depth limit 100.

The critical property for LM Software: the system's 448 builtin primitives are Prolog predicates. kb_assert, queue_push, counter_increment, file_read, json_parse — every operation the system can perform is callable from Prolog. When a user writes a Prolog rule, they are writing a program that chains system operations together. This is programming — control flow, conditionals, data transformation, I/O — expressed as logic rules over typed predicates with exact arithmetic.

Rules compose primitives into new operations. A rule at root scope is permanent and available to everything. A rule at session scope is disposable and dies with the session. A rule at project scope persists across sessions within that project. Properties of composed rules — pure, deterministic, partial — derive from the component primitives.

2.4 Sessions, Snapshots, and Clones

The system cleanly separates persistent state (facts, rules, constraints — survives everything) from live state (data primitives, working data, scope — cleared by reset, captured

by snapshot) [VDR-8, VDR-14].

A snapshot atomically captures all live state. Typical size: 10-500 KB. Small because it captures state, not knowledge — the knowledge is in persistent KBs that don't need to be in the snapshot.

A clone forks an independent copy from a snapshot. Clones share persistent KBs (visible to all, read-only for shared portions) and have independent live state. When a clone is killed, only its live state is destroyed. Any facts it committed to persistent KBs through the normal pipeline — with grants, constraints, and provenance — survive.

Sibling clones cannot see each other. This is structural: the scope walk algorithm goes up to ancestors and down into children, never sideways to siblings. Cross-clone data leakage is not prevented by access control — it is prevented by the absence of any path between siblings in the tree.

---

3. KBs as Machines

Each KB is not merely a data container. It is a machine with full mechanical ability.

A KB has state: counters that track quantities with bounds and clamping, queues that buffer messages in FIFO order, stacks that manage LIFO work items, ring buffers that hold rolling time-series data with automatic overwrite, bitsets that track completion of numbered items, LRU caches that store recent results with automatic eviction, and locks that provide non-blocking coordination signals.

A KB has logic: Prolog rules stored as persistent facts that fire when queried, evaluating conditions and chaining builtin operations.

A KB has data: facts at integer addresses with provenance showing when they were created, by whom, through what operation, from what source.

A KB has constraints: axiom constraints that can never be suspended (sum-to-one, audit immutability), operational constraints that can be suspended with logging (rate limits, capacity bounds), legal constraints that activate by jurisdiction, and project constraints configured by users.

A KB has connections: typed directed relationships to other KBs, enabling the machine to reference and interact with other machines.

A KB has visibility: public (all users), internal (operators and owners), or owner-only (owning entity). An integer comparison, not an LLM judgment.

A KB has grants: positive credentials that authorize specific operations. No grant means the operation is denied before any logic executes.

When a user builds an LM Software application, they are wiring machines together. A queue at path A, a processor watching path A, results written to path B, a downstream runner watching path B. The topology of machines in the tree is the application architecture. The tree is the wiring diagram.

4. Prolog as the Programming Language

4.1 Builtins Are Predicates

The system provides 448 builtin primitives organized into 25 categories [VDR-6, VDR-10, VDR-14]. 404 are pure — no side effects, deterministic, bounded. 44 are operational — they touch the filesystem, network, or processes, and require positive credential grants.

Every builtin is a Prolog predicate. This means a user-written Prolog rule can call any builtin directly:

```
process_intake(SourceQ, TargetKB) :-

queue_pop(SourceQ, Message),

json_parse(Message, Data),

kb_assert(TargetKB, finding(Data)),

counter_increment(SourceQ, processed_count).
```

This rule pops a message from a queue, parses it as JSON, asserts the parsed data as a fact in a target KB, and increments a counter. Four builtin calls chained in one rule. When this rule fires, Prolog executes it. The LLM is not involved in execution — it wrote the rule during development, and now the rule runs on its own.

4.2 Three Execution Levels

LM Software operates at three levels of automation, and applications move from the first toward the third as they mature:

Level 1 — Full LLM judgment. The LLM decides every step at runtime. It reads state from KBs, chooses which primitive to invoke, formulates the call, interprets the result. 50-500 tokens per task. This is how applications begin — during development, everything requires judgment.

Level 2 — LLM invoking stored Prolog. The LLM recognizes that a situation matches a stored rule and invokes it. 8 tokens per invocation — one command token to call the rule. Prolog executes the chain of builtins. The LLM provided judgment on when to call; the execution is deterministic.

Level 3 — Pure Prolog batch. Zero LLM tokens. Rules fire on triggers or schedules. Builtins execute. Results land at known addresses. The LLM is not involved at all. This is fully automated operation — the application runs as stored programs.

4.3 Dynamic Arguments

Rules do not need hardcoded paths. They take arguments:

```
transfer_results(Source, Target, Filter) :-

query_in(Source, finding(X)),

Filter(X),

kb_assert(Target, verified_finding(X)).
```

The logic is static — query, filter, assert. The data is dynamic — which source, which target, which filter. The same rule serves dozens of workflows by being pointed at different KBs.

Arguments can come from three places: the LLM provides them at invocation time (8 tokens); a configuration KB holds them as facts that the rule queries at runtime (zero LLM tokens); or they are inherited through the scope chain from a parent KB's working data.

4.4 Mixed Execution

Prolog rules can invoke Python scripts through the execute_python builtin predicate. The script runs in a sandboxed environment, writes output to a KB location, and the next Prolog rule picks up the output. Prolog orchestrates, Python computes, builtins connect them. The LLM wrote all three during development. At runtime, the chain executes without LLM involvement unless a genuinely novel situation arises that no existing rule covers.

---

5. The Development Lifecycle

5.1 Development Is Conversation

The user interacts with an LLM session. They describe what they want the application to do. They provide sample inputs. They load data into KBs — product information, policy documents, runbooks, reference data. They test the session against scenarios and observe its behavior.

When the session handles a case correctly through LLM judgment, the user or the LLM encodes that resolution as a Prolog rule. The pattern is captured. Next time the same situation arises, the rule fires and the LLM doesn't need to reason through it again.

When the session handles a case incorrectly, the user corrects it. The correction might be a new rule, a modified rule, an additional constraint, or a corrected fact. The fix is immediate — one primitive call and the correction is live.

This is iterative development. The user is building software through a conversation, testing it interactively, and encoding working behavior as reusable rules. The conversation is the IDE.

5.2 Testing Is Interactive Verification

The user runs test cases against the session and verifies outputs. Test cases can be informal — "what's our return policy for opened items?" — or structured — feeding sample data through the processing pipeline and checking the output facts.

Because every computation is exact and every fact has provenance, verification is precise. The user can query the provenance chain of any output to see exactly which rules fired, which facts were consulted, which builtins were invoked, and what confidence the system assigns to each step. There is no black box. The entire derivation is inspectable.

Rules that pass testing are promoted from session scope (disposable) to project scope (persistent across sessions) or higher. Rules that fail are retracted — cleanly, completely, with no residual effect on other rules.

5.3 Building Is Snapshotting

When the session handles its intended workload correctly, the user snapshots it. The snapshot atomically captures all live state: loaded rules, mounted KBs, scope configuration, data primitive states, working data bindings.

The snapshot is the binary. It encodes everything the session learned during development — which rules to apply, what scope to operate in, what grammars to use, what constraints to enforce. It does not contain the persistent KB data it references — that data stays in the tree, shared across all clones that reference it.

Multiple snapshots of the same application can coexist, representing different versions. Snapshot v1 and v2 and v3 are separate artifacts. Rolling back is deploying clones from an earlier snapshot. Comparing versions is a structural diff on two sets of facts.

5.4 Deployment Is Cloning

Every deployment is a clone of a snapshot. The clone inherits all live state from the snapshot and has independent execution from that point forward.

Deployment patterns depend on the application type. Clone-per-request: each incoming user gets a fresh clone that dies when the interaction ends. Clone-per-task: each work item gets a fresh clone that dies when the task is complete. Long-lived clone: a persistent instance that runs until a drift threshold fires, then is killed and replaced with a fresh clone from the same snapshot.

Drift management ensures clones stay healthy. Counters track turns elapsed, context saturation, denominator drift, and error rate. When any metric exceeds its threshold, the clone is killed and a fresh one is spawned from the frozen snapshot. The snapshot never degrades. Freshness is guaranteed by policy, not by hope.

5.5 Monitoring Is Polling

Polling runners — lightweight LLM instances that spawn fresh on a timer, check conditions, route work, and die — provide monitoring. They read counters and queue depths and bitsets across the application's KBs. They flag anomalies, spawn replacement workers, alert on threshold violations. 10-50 tokens per cycle. Fresh LLM every cycle — no attention degradation, no accumulated confusion.

5.6 Updating Is Changing Facts

When a policy changes, the user changes a fact in the persistent KB. Every future clone picks up the change immediately because persistent KBs are shared read-only. No retraining. No redeployment. No prompt template update. One fact at one integer address.

When a rule needs modification, the user retracts the old rule and asserts the new one. The retraction is clean — the old rule is gone, its provenance record shows it was retracted and when. Downstream facts derived solely from the retracted rule can be identified through the provenance chain and flagged for review.

5.7 Retirement Is Archival

When an application is no longer needed, its snapshot KB is archived — frozen but queryable. All KBs it created, all facts it asserted, all rules it wrote, all provenance records are preserved. The application's complete history is available for audit indefinitely. Retirement is a metadata change, not a deletion.

---

6. Runner Types as Application Classes

The VDR system defines four runner types [VDR-20], each a different class of LM Software application:

6.1 Interactive Runners

Activated by user input. Inherit the authenticated user's grants. Reactive — idle between interactions. 50-500 tokens per activation. These are user-facing applications: customer support chatbots, investigation assistants, analysis tools, interactive dashboards.

The user developed the application by configuring an interactive session. Deployed it by making the snapshot available for clone-per-user instantiation. Each user gets a fresh clone. The clone handles the conversation, writes session-specific data to its session KB, and dies when the user disconnects.

6.2 Polling Runners

Timer-driven. Spawn fresh each cycle from a snapshot. Check queues, counters, directory watch lists. Route work. Terminate after the cycle completes. 10-50 tokens per cycle.

These are monitoring and routing applications: queue depth monitors, worker pool managers, health checkers, scheduled report generators. The user developed a session that knows how to check conditions and take actions, snapshot it, and put it on a timer.

6.3 Processor Runners

Maintain persistent data connections — streams, webhooks, file watchers. Long-lived with periodic respawn at configurable turn thresholds. 8-30 tokens per item processed.

These are data pipeline applications: log processors, document compactors, metric collectors, event handlers. The user developed a session that knows how to handle a specific data type, snapshot it, and pointed it at an input source.

6.4 Internal Processing Runners

Self-directed. Evaluate KB state on schedule. Run consistency checks, compute derived facts, identify coverage gaps, produce coverage metrics. Read-broad, write-derived-only. 20-100 tokens per activation.

These are self-maintenance applications: consistency validators, coverage trackers, rule quality monitors, derived fact generators. The user developed a session that knows how to evaluate system health, snapshot it, and scheduled it.

---

7. Inter-Application Communication

LM Software applications communicate through data primitives in the KB tree — the same primitives that give each KB its mechanical ability. Applications do not communicate directly. They read and write shared state at known addresses, decoupled by the data structures between them.

7.1 Queues for Message Passing

A queue is a bounded FIFO on a KB. Push to back, pop from front. Atomic operations — one push, one pop, separate read and write positions, no contention between producers and consumers.

Application A pushes a task description to root.projects.intake.queue. Application B pops from that queue, processes the task, pushes results to root.projects.output.queue. Application C pops from the output queue and aggregates results. The three applications do not know each other exist. They communicate entirely through queue state at known addresses.

Push returns false when the queue is full. This is backpressure — the producing application can react by slowing down, alerting a supervisor, or waiting for capacity. The queue is bounded by declared capacity, enforced on every mutation.

7.2 Counters for Coordination

A counter is a signed integer with min/max bounds that clamps rather than wrapping. Increment, decrement, add, get, reset, set. One atomic operation per mutation.

Counters serve multiple coordination roles. A worker count tracks active instances — each worker increments on spawn, decrements on death. A supervisor reads the counter and maintains the pool within bounds. A throughput counter tracks items processed per hour — a monitor compares against a minimum threshold and flags underperformance. A drift counter on a clone increments every turn — when it hits the threshold, the clone dies and is replaced. A grant-use counter decrements on every operational primitive invocation — when it hits zero, the grant is exhausted.

7.3 Bitsets for Synchronization

A bitset is a fixed-width bit array. Set, clear, test — one bitwise operation each.

Ten workers each processing a partition set their bit when done. A supervisor reads the bitset each cycle. When all 10 bits are set, the supervisor proceeds to the aggregation phase. This is barrier synchronization — all workers must complete before the next phase — implemented as one integer read and one comparison.

An evidence bitset in an investigation tracks which pieces of evidence have been evaluated. An endpoint bitset in an SRE investigation tracks which of 200 endpoints have been checked.

7.4 Locks for Signaling

A lock is a non-blocking coordination flag. Acquire, release, check. It never blocks — checking a held lock returns false, and the checking application decides what to do.

Application A acquires a lock on a KB before writing a batch of related facts. Application B checks the lock before reading — if held, skip and come back next cycle. If released, read the consistent batch. Cooperative signaling without deadlock risk.

7.5 Ring Buffers for Streaming State

A ring buffer is a fixed-size circular buffer. Write overwrites the oldest entry when full. No coordination needed between writer and reader.

A processor writes per-minute metrics to a ring buffer sized for 60 entries — always the last hour of data without unbounded growth. A monitor reads the ring each cycle to compute rolling averages and detect trends.

7.6 LRU Caches for Memoization

An LRU cache is a bounded key-value store with timestamps. Evicts least recently used when full.

A worker caches parsed results for deduplication — if the same input appears twice, the LRU catches it. An investigation caches attempted approaches with failure reasons — before trying a new approach, the system checks the LRU to avoid repeating known failures.

7.7 Topology Patterns

The user selects the communication topology by placing data primitives at paths and configuring runners to operate on them:

Waterfall: Queue A → Runner B → Queue C → Runner D → Queue E. Strictly sequential, fully decoupled. Each stage is independently deployable and replaceable.

Fan-out: One queue, multiple consumers. Each consumer pops atomically — whoever pops first gets the task. Work distribution without explicit load balancing.

Fan-in: Multiple producers, one queue, one consumer. The consumer aggregates results from all producers.

Peer: Shared queue, any runner can push, any runner can pop. Collaborative processing where any instance can handle any task.

Supervisor: One runner monitors counters and bitsets, spawns workers when needed, kills workers when drift thresholds fire, adjusts pool size based on queue depth. The supervisor's spawn authority is governed by its grants — it can create clones from snapshots it has access to, nothing more.

---

8. Worked Example: Customer Support Chatbot

8.1 The Problem

A company needs tier 1 customer support. The support agent must know the product catalog, pricing, return policy, business hours, contact information, escalation procedures, and common troubleshooting steps. All answers must be factually correct. No hallucinated phone numbers. No invented policies. No cross-customer data leakage between conversations.

8.2 Development

The user starts an interactive session. They load product data into KBs:

Product catalog facts at root.products.myapp.catalog — SKUs, names, descriptions, prices, availability. Each fact with provenance showing the source and load date.

Return policy as Prolog rules at root.products.myapp.policies:

```
return_eligible(Order, yes) :-
```

```
days_since_purchase(Order, D), D =< 30,

condition(Order, unopened).

return_eligible(Order, no) :-

days_since_purchase(Order, D), D > 30.

return_eligible(Order, no) :-

condition(Order, opened), category(Order, software).
```

Contact information as facts: contact_phone(support, “1-800-555-0199”), business_hours(weekday, 9, 17), business_hours(weekend, closed).

Escalation rules:

```
escalate(Ticket, tier2) :-

attempts(Ticket, N), N >= 3, unresolved(Ticket).

escalate(Ticket, tier2) :-

customer_sentiment(Ticket, angry), unresolved(Ticket).
```

Troubleshooting runbooks as Prolog rules that walk decision trees:

```
troubleshoot(cannot_login, Step1) :-

ask_customer(password_reset_attempted, Response),

next_step(cannot_login, password_reset_attempted, Response, Step1).
```

The user tests against scenarios. “Can I return my copy of ProductX that I bought 25 days ago and haven’t opened?” The rules evaluate: 25 =< 30, condition is unopened, result is yes. The LLM phrases the response in natural language. The policy was never in the token stream to be hallucinated — it was evaluated by Prolog and the result was handed to the LLM.

“What’s your phone number?” The LLM queries contact_phone(support, X). The fact returns “1-800-555-0199”. The LLM includes it in prose. The number came from a fact at a known address, not from token prediction.

The user iterates. Edge cases that the rules don’t cover trigger LLM judgment. When the judgment is correct, the user encodes it as a new rule. The rule set grows. The LLM handles less and less through judgment and more and more through rule evaluation.

8.3 Building

When the session handles the full range of tier 1 scenarios correctly, the user snapshots it. The snapshot captures: loaded rules, mounted product KBs, scope configuration, greeting templates, escalation thresholds. Size: perhaps 50 KB.

8.4 Deployment

Every incoming customer triggers a clone from the snapshot. The clone starts with identical state — same rules, same product knowledge, same policies. The customer's conversation creates session-specific facts in the clone's session KB: what they asked about, their order data, their ticket status. When the conversation ends, the clone dies. The session KB is destroyed. The product KBs and policy rules are untouched. The next customer gets another fresh clone.

A thousand concurrent customers means a thousand independent clones. No cross-customer contamination — structurally impossible because sibling clones cannot see each other's session KBs.

8.5 Updating

The return policy changes from 30 days to 45 days. The user modifies one rule in root.products.myapp.policies — retract the old rule, assert the new one with D =< 45. Every future clone picks up the change immediately because the policy KB is persistent and shared read-only. No retraining. No redeployment. One retraction and one assertion.

A new product launches. The user asserts new facts in the catalog KB. Every future clone sees the new product. The snapshot doesn't need to change because the snapshot doesn't contain the catalog — it contains a reference to the catalog KB.

8.6 Monitoring

A polling runner checks customer support metrics every 5 minutes: queue depth for waiting customers, average resolution time (from counters), escalation rate (from counters), customer satisfaction (from facts asserted at conversation end). If escalation rate exceeds threshold, the poller flags it for human review.

8.7 Accumulation

Over hundreds of conversations, the chatbot encounters new questions that require LLM judgment. Each time the judgment produces a correct answer, it can be encoded as a rule. After a month of operation, the rule set covers 80% of incoming questions at level 3 (pure Prolog, zero LLM tokens). The remaining 20% require LLM judgment — genuinely novel questions, ambiguous situations, edge cases. The application improved through usage without any rebuild.

8.8 Contrast with Conventional Approach

A conventional chatbot uses a prompt template with instructions ("You are a helpful customer support agent for ProductX…"), RAG retrieval of product documents pasted into context, and the LLM generating every answer from scratch. The phone number might be hallucinated. The return policy might be paraphrased incorrectly. Two concurrent users share the same model instance with no state isolation. Updating the return policy requires editing a document, re-indexing the retrieval system, and hoping the model interprets the new document correctly. There is no audit trail for individual answers. There is no way to verify which policy was applied to a specific customer interaction. Accumulation is impossible — every conversation starts from zero.

---

9. Worked Example: File Processing Pipeline

9.1 The Problem

An organization receives hundreds of technical documents daily — incident reports, vendor assessments, compliance filings, research papers. Each must be classified, compacted into structured form, stored with provenance, and indexed for query. The processing must scale with volume and improve with experience.

9.2 Development

The user develops a file processor session. They teach it to handle the first document type — say, SRE incident reports. The session learns to extract key entities (affected service, root cause, resolution, timeline), encode relationships (cause → effect, service → dependency), and produce structured facts with provenance.

The extraction logic becomes Prolog rules:

```
extract_entity(Doc, service, Name) :-

field(Doc, affected_service, Name).

extract_entity(Doc, root_cause, Cause) :-

field(Doc, root_cause_analysis, Text),

classify(Text, cause_category, Cause).
```

Compaction grammars become persistent KB fields — templates that strip prose while preserving every named concept, relationship, and claim [VDR-12]. The grammar handles structural tokens (pipes, headers, delimiters) at zero cost. The LLM fills content slots only.

The user tests against sample documents, verifies the output facts match expectations, corrects errors, adds rules for edge cases. When SRE incident reports are handled correctly, they move on to vendor assessments, adding rules for that document type. Each type adds rules; previously working types continue working because rules at different scopes don't interfere.

9.3 Building the Worker

When the session handles all target document types, the user snapshots it. This is the file processor binary.

9.4 Deploying the Pipeline

The pipeline consists of multiple applications wired through the KB tree:

Intake queue at root.projects.docprocessing.intake — a bounded FIFO where new documents arrive. An external script or a processor runner watching a directory pushes file references to this queue.

50 worker clones from the processor snapshot. Each pops from the intake queue (atomic pop — whoever pops first gets the task), processes the document, asserts structured facts to root.projects.docprocessing.findings, increments a processed counter, and moves to the next item. Each worker has a drift threshold of 200 turns. When the counter hits 200, the worker dies.

3 polling clones monitoring the system. Every 30 seconds, each poller spawns fresh from its snapshot, checks the intake queue depth, checks the active worker count (from a counter), checks the dead worker count (from a counter). If active workers are below minimum, the poller spawns replacements from the frozen worker snapshot. If intake queue depth exceeds a threshold, the poller spawns additional workers. If error rate (from a counter ratio) exceeds 5%, the poller flags the system for human review. Then the poller dies. Fresh every cycle.

1 internal processing runner evaluating coverage. On a schedule, it scans the findings KB for consistency — checking that every document has all required entity types extracted, that relationship chains are connected, that provenance is complete. It produces a coverage metric as a VDR fraction with a remainder that specifically identifies what is missing. Coverage gaps become queryable facts that can drive further processing.

9.5 Pipeline Communication

The workers and pollers communicate entirely through KB state. A worker pops from a queue, writes to a KB, increments a counter. A poller reads counters and queue depths. A supervisor spawns and kills workers based on threshold comparisons. No direct communication between any runners. Decoupled processes coordinating through shared data structures at known integer addresses.

The output KB at root.projects.docprocessing.findings accumulates structured facts from all workers. A downstream application — perhaps an interactive runner for analysts —

queries this KB to find patterns, compare documents, trace relationships. The processing pipeline and the analysis application are separate LM Software applications communicating through persistent KB state.

9.6 Accumulation

By document 50, the processor has encountered most common structures for each document type. Compaction rules handle them automatically. By document 200, the system reaches roughly 98% fact parity with manual processing [VDR-19]. The LLM is invoked only for genuinely novel document structures — new types that no existing grammar or compaction rule covers.

When a novel structure appears, the LLM handles it through judgment (level 1), then encodes the resolution as a new compaction rule (promoting to level 3 for future documents of that type). The rule set grows. The per-document token cost drops. The workers handle more documents per drift cycle because each document requires fewer tokens.

9.7 Scaling

More documents per day? Increase the worker pool by spawning more clones from the same snapshot. The snapshot doesn't change. The pollers detect queue depth increases and can auto-scale the pool within configured bounds.

New document type? Develop a new processor session, add rules for the new type, snapshot it. Either update the existing worker snapshot to include the new rules or deploy a separate worker pool for the new type with its own intake queue.

The processing pipeline is LM Software — developed through conversation, deployed as snapshots, scaled by cloning, improved by usage, governed by structure.

---

10. Copy-on-Write and Versioning

10.1 Clone Destinations

When a user clones a snapshot, the destination path determines everything about the clone's lifecycle, visibility, and governance. The clone mechanism is identical regardless of destination — the path determines the rules.

Personal scratch: root.users.jane.scratch.myapp_experiment. Owner-only visibility. Jane's grants. Full freedom to experiment. Disposable. If she discovers something useful, she promotes specific facts back to the source tree through the standard pipeline — grants, constraints, provenance, audit.

Team experiment: root.org.acme.engineering.experiments.new_caching_layer. Internal visibility. The team's grants. Department constraints inherited and enforced. Multiple engineers can see and contribute. Results promote to production or archive with full provenance of what was tried.

Production deployment: root.products.myapp.deployments.v3. Org-wide constraints. Audit requirements inherited as axiom constraints. Frozen configuration after deployment — an axiom constraint prevents modification of deployed state.

Ephemeral session: root.sessions.sess_4827. Exists for the duration of the session. Everything in it is live state. The session ends, the clone dies, zero residue. Committed facts survive in persistent KBs; session state vanishes.

10.2 Destination Governance

Rules governing where clones can be placed are themselves KB facts at the appropriate scope. An organization can require that experimental forks of production builds go under the department's experiments branch, never under personal scratch. This rule fires when someone attempts to clone a production snapshot to a personal path — the constraint evaluates, the clone is redirected or blocked. The policy is structural, not documented. Nobody has to remember it. Nobody can forget it. Nobody can override it without an admin grant.

10.3 Versioning

Multiple snapshots of the same application coexist as sibling KBs:

root.products.myapp.snapshots.v1

root.products.myapp.snapshots.v2

root.products.myapp.snapshots.v3

Diffing two versions is a structural operation — compare two sets of facts at known addresses. The diff itself becomes a KB: which rules were added, which were modified, which were removed, with provenance on each change.

Rollback is deploying clones from an earlier snapshot. Canary deployment is running clones from two snapshots simultaneously and comparing results — each clone writes to its own output path, a supervisor compares output facts, exact VDR fraction comparisons determine whether the new version is performing within tolerance.

---

11. Negative Accumulation and Hygiene

Accumulation is not always positive. Rules go stale — the infrastructure changes, APIs move, correlation patterns become obsolete. Scripts break when their target environment changes. Grammars stop matching when input formats evolve.

11.1 Detection

Every data primitive supports hygiene monitoring. The LRU cache on a rule set tracks last-fired timestamp and hit count per rule. Counters track success rate per rule — how often a rule fires and produces a result that downstream logic accepts versus rejects.

Prolog rules at the operational scope automate detection:

```
stale_rule(RuleID) :-

last_fired(RuleID, Timestamp),

days_since(Timestamp, D), D > 90.

failing_rule(RuleID) :-

fire_count(RuleID, Fires), Fires > 10,

success_count(RuleID, Successes),

Ratio is Successes / Fires, Ratio < 20/100.

permission_orphan(RuleID) :-

grant_denial_count(RuleID, Denials), Denials > 0,

grant_success_count(RuleID, Successes), Successes = 0.
```

11.2 Remediation

Stale rules are flagged for review. If not manually promoted within a configurable window, they are retracted. Failing rules are retracted immediately when their success rate drops below threshold. Permission orphans — rules that have never successfully executed because they lack the required grants — are flagged and retracted if no grant is forthcoming.

Retraction is clean. The fact is removed from the KB. Downstream facts derived solely from the retracted rule are identified through the provenance chain and flagged for review. No weight matrix pollution. No wondering whether stale knowledge is still influencing outputs somewhere invisible.

11.3 Contrast with Conventional Systems

Stale knowledge in conventional LLM weights is permanent, unidentifiable, and irremediable. Fine-tuning against stale knowledge adds new weight adjustments on top of the old ones. There is no mechanism to find stale knowledge, no mechanism to remove it, and no way to verify that removal was complete. In LM Software, every piece of accumulated knowledge is individually addressable, inspectable, and retractable. Hygiene is a maintenance task, not an impossibility.

---

12. Security Model

12.1 Security Is Structure

LM Software applications inherit all security properties from the KB tree architecture [VDR-16, VDR-14]. Security is not configured separately from the application — it falls out of where things are placed in the tree and what grants exist.

Scope: Each runner operates at a position in the tree. Its scope walk reaches its own KB, its ancestors to root, and its children. Sibling branches are structurally unreachable. A customer support clone for Customer A cannot access Customer B's session because they are siblings, and the scope walk never traverses siblings.

Visibility: Each KB has a visibility level — public, internal, or owner-only. The check is an integer comparison inside the primitive that performs the query. If visibility fails, the KB is skipped entirely — facts absent, not redacted.

Grants: Every operational primitive requires a positive credential grant. No grant, the operation is denied before any logic executes. Grants specify: operation class, allowed operations, location constraints, issuer, expiration, maximum uses. Grant state transitions are monotonic — active can become expired, exhausted, or revoked, but never re-increment.

Constraints: Axiom constraints cannot be suspended. Children can tighten parent constraints but never loosen them. Constraint evaluation is exact integer arithmetic — no tolerance, no "close enough."

Audit: Every query attempt — granted or denied — is logged in an append-only audit KB protected by an axiom constraint against retraction.

12.2 Self-Generated Rules

When an LM Software application writes a Prolog rule during operation, that rule inherits all security properties. It goes through the command token pipeline. It is checked against grants. It is subject to constraints. It is logged with provenance.

A rule the LLM writes cannot grant itself elevated access — grant assertion requires an admin-level grant that runners do not possess. A rule querying out-of-scope data stores successfully but fires with empty results — scope checks happen at query time per fact, not at rule definition time. A rule attempting to copy restricted data to a public KB fails — both read grants on the source and write grants on the destination are checked independently.

The security model does not need to be extended for LM Software. It is the same model that governs all data access in the system. Self-extension is safe because every self-generated artifact passes through the same structural checks as every other artifact.

---

13. Economics of LM Software

13.1 Development Cost

LM Software development is measured in hours of interactive sessions, not weeks of engineering. A customer support chatbot: 4-8 hours of conversation to load data, write rules, and test scenarios. A file processing pipeline: 8-16 hours to handle a set of document types. A monitoring application: 2-4 hours to configure checks and thresholds.

Compare: conventional software development for the same applications requires a developer writing orchestration code, retrieval pipelines, state management, error handling, deployment configuration, and monitoring infrastructure. Weeks to months of engineering time.

13.2 Operational Cost

LM Software operational cost decreases over time. Rule formalization costs 25-40 tokens. Each reuse of that rule replaces 150-300 tokens of conventional LLM reasoning [VDR-19]. By the 5th reuse, amortized cost is under 10 tokens per use. At organizational scope with thousands of reuses, cost per use approaches zero.

The accumulation curve from the SRE case study [VDR-19]: investigation 1 costs 329 command tokens. Investigation 2 costs 42% less due to accumulated rules. Investigation 10 costs 72% less. Investigation 50 costs 88% less. Investigation 100 costs 93% less with 150 rules auto-firing.

Conventional LLM applications have flat or increasing operational cost — every conversation starts from zero, and longer conversations cost quadratically more due to attention over growing context.

13.3 Update Cost

Policy change: one fact retraction plus one fact assertion. Immediate effect. Zero retraining. Zero redeployment.

New product data: fact assertions in the catalog KB. Every future clone sees the updates immediately.

New document type: one development session to add rules, one snapshot update. Existing functionality unaffected.

Compare: conventional LLM application updates require editing prompts or documents, re-indexing retrieval systems, potentially retraining or fine-tuning, redeploying, and validating that the update didn't break existing behavior.

13.4 Scaling Cost

Additional capacity: spawn more clones from the same snapshot. The snapshot cost is fixed. Marginal cost per clone is the live state duplication (10-500 KB) plus the LLM tokens for the clone's work.

Geographic distribution: snapshots are portable. Clone in any datacenter that has access to the persistent KB tree.

---

14. Memory and Storage Efficiency

14.1 Structured, Not Prose

In a conventional LLM, state is prose in the context window — "as we discussed, the counter is at 7 and we've checked 3 of 5 endpoints and the threshold is 10." That sentence costs tokens to generate, tokens to re-read on every subsequent turn, and grows the context window quadratically.

In LM Software, the counter is an integer at a known offset. The threshold is a fact. The progress is a bitset. Three values at known addresses, totaling perhaps 20 bytes. No prose. No tokens. No context window growth.

14.2 Interned, Not Repeated

Every string in the system is interned — assigned an integer ID on first encounter, and every subsequent reference uses the integer. The predicate name "deployment_correlates_with" is not stored as a 30-character string everywhere it appears. It is stored once in the intern table and referenced by a 4-byte integer everywhere else. Comparison is integer equality, not string matching.

14.3 Typed, Not Serialized

Enums are small integers indexing into tables. Status values, severity levels, constraint types, visibility levels — each is 1-4 bytes, not the string "critical" or "owner_only" repeated in memory. Scope chains are stack-allocated arrays of 16 i32 values — 64 bytes total, no heap allocation, no pointer chasing [VDR-11].

14.4 Scaling Through Hierarchy

The KB tree scales by adding branches and leaves, not by making anything bigger. An organization with 50 KBs and one with 500,000 KBs use the same scope walk — depth bounded at 16 levels, width irrelevant. Each user's path through the tree is the same depth regardless of how many siblings exist. Scope walk cost depends on tree depth, not tree size. Width is free. Depth is bounded.

A conventional context window scales linearly with knowledge — more knowledge, more tokens, more cost per turn. A KB tree scales hierarchically — more knowledge, more branches, same traversal cost per query.

---

15. Limitations

15.1 LLM Judgment Bounds Quality

The application is only as good as the LLM judgment that shaped it. Poorly developed sessions produce poor applications — wrong rules, missing edge cases, inappropriate escalation logic. LM Software shifts the bottleneck from developer skill to user judgment, but does not eliminate it.

15.2 Development Investment

The gap between level 1 (full LLM judgment) and level 3 (pure Prolog execution) is the development investment. Not every workflow reaches level 3. Some domains have too much variety for rule coverage to reach high percentages. The user must judge when the investment in additional rules is worth the reduction in per-task token cost.

15.3 Snapshot Growth

Live state grows as rules accumulate. A session that has written 500 rules has a larger snapshot than one with 50. Snapshot size remains small (typically KB-range, not MB) because rules are structured — predicate ID, argument types, body goal references — not prose. But very large rule sets increase clone startup time proportionally.

15.4 Novel Inputs Require Tokens

Pure Prolog execution handles only cases covered by existing rules. Novel inputs — new question types, unexpected document structures, unprecedented scenarios — fall through to LLM judgment and consume tokens. The accumulation curve flattens but never reaches zero LLM cost for applications that face genuinely diverse inputs.

15.5 Not for All Workloads

LM Software is not appropriate for compute-intensive numerical work, real-time control, hardware interfacing, graphics, or domains where conventional compiled software provides deterministic performance guarantees that LLM-based execution cannot match. LM Software targets judgment-heavy, policy-governed, data-rich workflows where the value is in correctly applying rules to varied inputs.

Appendices

Appendix A — Customer Support Chatbot: Complete Walkthrough

A.1 KB Tree Layout

```
root.products.myapp
```

```
├── catalog (facts: SKUs, names, prices, availability)
├── policies (rules: return, warranty, shipping, pricing)
├── support
│   ├── contact (facts: phone, email, hours)
│   ├── runbooks (rules: troubleshooting decision trees)
│   ├── escalation (rules: escalation criteria)
│   └── templates (grammars: greeting, closing, apology, confirmation)
└── deployments
└── support_v3 (snapshot: the deployed chatbot application)
```

A.2 Development Session Transcript (Abbreviated)

Turn 1: User loads product catalog from CSV. System parses, asserts ~200 facts to catalog KB with provenance.

Turn 2: User provides return policy document. System compacts to Prolog rules, user verifies.

Turn 3-5: User tests against scenarios — "can I return X," "what's your phone number," "my product doesn't work." System handles each through rule evaluation plus LLM prose.

Turn 6-8: User identifies edge cases — partial refunds, warranty versus return, international shipping. Writes additional rules.

Turn 9: User verifies escalation logic — angry customers, repeated failures, billing disputes.

Turn 10: User snapshots the session as support_v3.

A.3 Clone Lifecycle

Customer connects → System spawns clone from support_v3 snapshot → Clone creates session KB at root.sessions.sess_NNNN → Customer asks question → Clone queries policy rules with customer data → Rules evaluate to result → LLM phrases result in natural language → Customer asks another question → Repeat → Customer disconnects → Clone dies → Session KB destroyed → Persistent KBs untouched.

A.4 Policy Update Procedure

Administrator retracts: return_eligible(Order, yes) :- …, D =< 30, …

Administrator asserts: return_eligible(Order, yes) :- …, D =< 45, …

Provenance records: old rule retracted by admin at timestamp, new rule asserted by admin at timestamp.

Effect: immediate for all future clones. Active clones continue with old rule until they die and are replaced.

Appendix B — File Processing Pipeline: Complete Walkthrough

B.1 KB Tree Layout

```
root.projects.docprocessing
├── intake (queue: incoming document references)
├── findings (facts: extracted entities, relationships, structured data)
├── rules (Prolog: extraction patterns per document type)
├── grammars (compaction templates per document type)
├── metrics
│   ├── throughput (counter: documents processed per hour)
│   ├── errors (counter: processing failures)
│   ├── worker_count (counter: active workers)
│   └── coverage (ring buffer: coverage scores per cycle)
├── workers
│   └── processor_v2 (snapshot: the worker application)
└── monitors
└── poller_v1 (snapshot: the monitoring application)
```

B.2 Worker Clone Lifecycle

Poller spawns worker from processor_v2 snapshot → Worker pops from intake queue (atomic) → Worker reads document from referenced path → Worker classifies document type using classification rules → Worker applies compaction grammar for that type → Worker asserts extracted facts to findings KB with provenance → Worker increments throughput counter → Worker increments personal turn counter → If turn counter < 200: pop next item → If turn counter >= 200: worker dies → Poller detects dead worker via worker_count counter → Poller spawns replacement from same snapshot.

B.3 Poller Cycle

Timer fires → Fresh clone from poller_v1 snapshot → Check intake queue depth → Check worker_count counter → Check error counter → If queue_depth > threshold AND worker_count < max: spawn additional workers → If error_rate > 5%: flag for human review → If worker_count < min: spawn replacements → Poller dies → Timer fires again → Fresh clone.

Appendix C — Runner Type Comparison

Property	Interactive	Polling	Processor	Internal
Trigger	User input	Timer	Data arrival	Schedule
Tokens per activation	50-500	10-50	8-30 per item	20-100
Lifecycle	Session duration	Single cycle	Long-lived, respawned	Single cycle
Grant scope	User's grants	System monitoring grants	Data pipeline grants	Read-broad, write-derived
Clone frequency	Per user	Per cycle	At drift threshold	Per cycle
Conventional equivalent	Web application	Cron job	Stream consumer	Background service

Appendix D — Data Primitive Usage Patterns

Need	Primitive	Mechanism	Example
Message passing	Queue	Atomic push/pop, bounded FIFO	Task distribution to workers
Progress tracking	Bitset	Set bit N when item N complete	200 endpoints, bit per endpoint
Throughput monitoring	Counter	Increment per item, read per cycle	Items processed per hour
Pool management	Counter	Increment on spawn, decrement on death	Active worker count
Drift detection	Counter	Increment per turn, kill at threshold	Clone freshness enforcement
Barrier sync	Bitset	All bits set triggers next phase	All workers done, begin aggregation
Deduplication	LRU cache	Check before processing, add after	Duplicate document detection

Need	Primitive	Mechanism	Example
Failed approach memory	LRU cache	Store failures, check before retrying	Investigation backtracking
Rolling metrics	Ring buffer	Fixed-size circular, auto-overwrite	Last 60 minutes of throughput
Batch consistency	Lock	Acquire before write, release after	Atomic multi-fact assertion
Rate limiting	Counter	Decrement per use, deny at zero	Grant use tracking
Authorization signal	Lock	Check before proceeding	Cooperative access coordination

Appendix E — Topology Patterns with KB Tree Layouts

E.1 Waterfall

```
root.pipeline
├── stage1_queue (queue)
├── stage1_worker (snapshot, pops from stage1_queue, pushes to stage2_queue)
├── stage2_queue (queue)
├── stage2_worker (snapshot, pops from stage2_queue, pushes to stage3_queue)
├── stage3_queue (queue)
└── stage3_worker (snapshot, pops from stage3_queue, writes to output KB)
```

E.2 Fan-Out

```
root.parallel
├── intake (queue, single producer)
├── workers (snapshot, N clones, all pop from same intake queue)
└── output (KB, all workers assert findings here)
```

E.3 Supervisor

```
root.managed
```

```
├── intake (queue)
├── workers (snapshot, pool of N clones)
├── metrics
│   ├── queue_depth (counter, read by supervisor)
│   ├── active_workers (counter, maintained by workers)
│   └── error_rate (counter ratio)
└── supervisor (snapshot, reads metrics, spawns/kills workers)
```

Appendix F — Snapshot Format and Size Estimates

Application type	Typical rules	Typical grammars	Live state	Estimated snapshot size
Simple chatbot	30-50	5-10	Minimal	20-50 KB
Full support bot	100-200	15-25	Moderate	50-150 KB
File processor	50-100	10-20	Queue refs, counters	30-80 KB
SRE investigator	150-300	20-30	Investigation state	80-200 KB
Monitoring poller	10-20	2-5	Counter refs	10-20 KB

Appendix G — Security Scenarios

Attack	Mechanism	Structural result	Reason
Clone reads sibling's session	Scope walk	Empty result	Siblings unreachable by walk algorithm
Runner elevates own grants	Grant assertion	Denied	Requires admin grant runner doesn't possess
Rule queries out-of-scope KB	Rule fires normally	Empty result set	Scope check at query time per fact
Malicious input injects rule	Compaction + assertion	Blocked by constraint	Axiom constraints evaluate rule content

Attack	Mechanism	Structural result	Reason
Data copy to public KB	Read + write	Denied	Both read grant (source) and write grant (target) checked independently
Cross-customer data in chatbot	Session fact query	Empty result	Each clone's session KB is independent branch

Appendix H — Accumulation Metrics Across Application Types

Application	Investigation 1 tokens	Investigation 10	Investigation 50	Investigation 100	Auto-triage at 100
SRE triage	329	92	65	55	93%
Support chatbot	200	80	50	40	85%
Document processing	150	60	35	25	92%
Compliance review	250	100	55	45	88%

Appendix I — Conventional Development Lifecycle Comparison

Phase	Conventional software	LM Software
Development	Write code in IDE	Interactive conversation
Testing	Unit tests, integration tests	Run scenarios, verify outputs
Building	Compile, package	Snapshot session
Deployment	Container orchestration	Clone snapshot
Monitoring	External dashboards	Polling runners reading counters
Updating	Code change, rebuild, redeploy	Change fact, immediate effect
Scaling	More containers, load balancers	More clones from same snapshot
Rollback	Redeploy previous container	Clone from earlier snapshot
Retirement	Decommission infrastructure	Archive snapshot KB

Appendix J — Comparison with Conventional LLM Applications

Property	RAG application	LangChain agent	LM Software
Orchestration	Developer-written code	Developer-written Python	Prolog rules in KB
State management	External database	External state store	KB facts at integer addresses
Memory across sessions	None (or developer-built)	None (or developer-built)	Automatic via rule accumulation
Policy enforcement	Prompt instructions	Code logic	Prolog rules + constraints
Factual accuracy	Depends on retrieval quality	Depends on tool output	Exact: facts at known addresses
Security	Application-level code	Application-level code	Structural: scope + visibility + grants
Improvement over time	Requires developer intervention	Requires developer intervention	Automatic via usage
Deployment	Application deployment pipeline	Application deployment pipeline	Clone snapshot
Update	Re-index, redeploy	Code change, redeploy	Change fact
Audit trail	If developer built one	If developer built one	Automatic: every operation logged
Developer required	Yes	Yes	No

[**HOWL-VDR-24-2026**] LM Software. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-24-2026

[**HOWL-VDR-1-2026**] VDR Arithmetic: Value, Decimal, Remainder. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-1-2026

[**HOWL-VDR-2-2026**] VDR Gym: Exact Arithmetic Across Fifteen Domains. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-2-2026

[**HOWL-MATH-3-2026**] The Transcendental Hierarchy. Github: https://github.com/ghowland/papers/tree/main/papers/M
MATH-3-2026

[**HOWL-MATH-4-2026**] The Universal Power-of-Two Basis. Github: https://github.com/ghowland/papers/tree/main/pape
MATH-4-2026

[**HOWL-VDR-14-2026**] VDR-LLM-Prolog. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL
VDR-14-2026

[**HOWL-VDR-21-2026**] VDR-LLM-Prolog on FPGA. Github: https://github.com/ghowland/papers/tree/main/papers/VD
VDR-21-2026

[**HOWL-VDR-22-2026**] VDR-LLM-Prolog on Dedicated Silicon. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-22-2026

[**HOWL-VDR-23-2026**] VDR-LLM-Prolog: Functional Remainder Hardware. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-23-2026